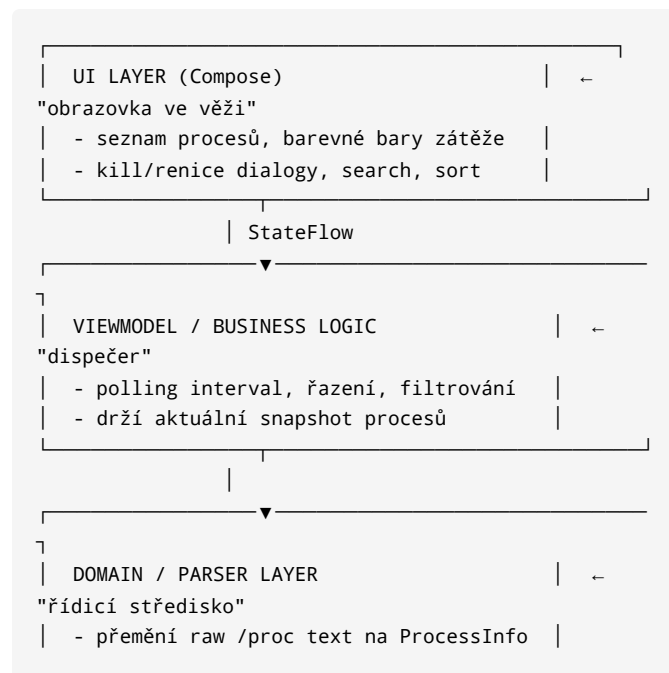
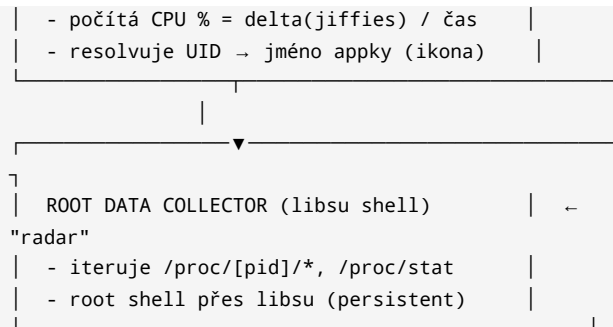


Rooted Android Task Manager — Architektura + Prompt pro agenta

Cíl: standalone nativní Android aplikace (Kotlin + Jetpack Compose), grafický htop-ekvivalent,
běžící přímo na telefonu (NE v proot, NE integrovaná do jiné existující appky), vyžadující root.

1. Architektura — 4 nezávislé vrstvy (analogie: řídicí věž letiště)





Proč root musí být: od Androidu 7+ (hidepid=2) běžná appka vidí ve `/proc` jen svůj vlastní

PID – "vidí jen svoje letadlo, ne celé nebe". Root shell tohle omezení obchází.

Výhoda oddělených vrstev: každou lze měnit nezávisle. Chceš přidat "zobrazit jen uživatelské appky" → mění se jen Domain/ViewModel vrstva, UI a Root Collector zůstávají netknuté.

2. Datový tok (jak se počítá CPU %, přesně jako u htop)

1. Collector přečte `/proc/stat` (celkové jiffies systému) a `/proc/[pid]/stat` (jiffies procesu)
– snapshot A.
2. Po `refreshIntervalMs` (výchozí 2000 ms) přečte znovu – snapshot B.
3.
$$\text{cpu\%} = \frac{(\text{proces_jiffies_B} - \text{proces_jiffies_A})}{(\text{celkem_jiffies_B} - \text{celkem_jiffies_A}) * 100 * \text{počet_jader}}$$

4. Bez tohoto kroku dostaneš jen "celkový čas běhu", ne aktuální zátěž — to je nejčastější chyba při psaní vlastního task manageru.
-

3. Funkce (feature list)

- Seznam procesů: PID, jméno, CPU %, RSS paměť, stav (R/S/D/Z), UID/vlastník
 - Rozpoznání jména a ikony Android aplikace z UID (přes `PackageManager.getPackagesForUid()`) — vylepšení oproti klasickému htop, který zná jen raw process name
 - Řazení podle sloupce (CPU, RAM, PID, jméno)
 - Vyhledávání/filtr (jen uživatelské appky vs. systémové procesy)
 - Barevné pruhy zátěže u CPU/RAM (jako htop)
 - Long-press / swipe akce: kill (SIGTERM → SIGKILL), renice
 - Nastavitelný refresh interval
 - Per-core CPU přehled (volitelně, druhá obrazovka)
 - Celkový přehled nahoře (celkové CPU, RAM, swap — jako hlavička htop)
-

4. Technický stack

- Kotlin, Jetpack Compose (Material 3)
- MVVM: `ViewModel` + `StateFlow` + `Coroutines` pro polling smyčku
- **libsu** (topjohnwu) — standard pro root shell na Androidu, persistentní shell (nepromptuje su při každém refreshi), Magisk-kompatibilní

- minSdk 26+, targetSdk aktuální stable
- Gradle (Kotlin DSL)

5. Struktura projektu

```

app/
├─ data/
│   └─ RootProcessDataSource.kt    (libsu shell
čtení /proc)
│       └─ ProcRawSnapshot.kt
├─ domain/
│   └─ ProcessInfo.kt              (model)
│   └─ ProcessParser.kt           (raw →
ProcessInfo, výpočet CPU %)
│       └─ AppLabelResolver.kt     (UID →
jméno/ikona appky)
├─ ui/
│   └─ ProcessListScreen.kt
│   └─ ProcessListViewModel.kt
│   └─ components/ (ProcessRow, LoadBar,
KillDialog)
└─ MainActivity.kt

```

6. PROMPT PRO AGENTA (zkopíruj a předej kódovacímu agentovi)

Postav standalone nativní Android aplikaci v Kotlinu (Jetpack Compose, Material 3) – grafický správce procesů/úloh (ekvivalent htop) pro ROOTED telefon. Je to samostatná appka, ne modul jiné existující aplikace, neběží v proot ani v Termuxu.

ARCHITEKTURA (dodrž striktní oddělení vrstev, MVVM):
 1. data/ – RootProcessDataSource: root shell přes

knihovnu `libsu` (com.github.topjohnwu.libsu),
persistentní root shell (negenerovat nový su
prompt při každém refreshi). Čte:
- /proc/stat (systémové CPU jiffies)
- /proc/[pid]/stat, /proc/[pid]/status,
/proc/[pid]/cmdline pro každý PID ve /proc
2. domain/ – Parser převádí raw text na
ProcessInfo(pid, name, uid, cpuPercent, rssKb,
state).
CPU % počítej jako deltu jiffies mezi dvěma snímky
děleno deltou celkových jiffies systému,
krát počet jader – NE jako kumulativní čas běhu.
AppLabelResolver mapuje UID přes
PackageManager.getPackagesForUid() na jméno a ikonu
appky,
pokud UID patří userspace aplikaci; jinak zobraz
raw process name.
3. ui/ – ProcessListViewModel drží
StateFlow<List<ProcessInfo>>, coroutine polling loop
s nastavitelným intervalem (default 2s), řazení a
filtrování v ViewModelu, ne v UI.
Compose obrazovka: LazyColumn se seznamem procesů,
hlavička s celkovým CPU/RAM přehledem,
barevný load-bar u každého řádku, search bar, sort
podle sloupce, long-press menu
(Kill -SIGTERM / Kill -9 / Renice), potvrzovací
dialog před kill.

POŽADAVKY:

- minSdk 26, Kotlin DSL Gradle
- Root oprávnění žádej přes libsu Shell.getShell()
při startu, ošetři stav "root nedostupný"
- Žádné síťové oprávnění, appka pracuje jen lokálně
- Vytvoř kompletní funkční Gradle projekt včetně
build.gradle.kts, AndroidManifest.xml
- Zkompiluj a ověř, že build prochází (uživatel
nebude kompilovat lokálně na telefonu) –
pokud máš přístup k CI/build prostředí, vygeneruj i
debug APK
- Okomentuj vrstvy tak, aby šly rozšiřovat nezávisle
na sobě (uživatel je architekt, ne

primárně programátor – chce rozumět, kde se co mění, ne psát kód sám)

Poznámka k předání agentovi: protože nekompiluješ lokálně na telefonu, v promptu je explicitně požadavek, aby agent build ověřil/spustil sám (CI nebo cloud build prostředí), ne jen vygeneroval zdrojové soubory.